Mess and Canteen Management System

DBMS Project Brief | Team of 6 | Version 1.0

1. Project Overview

The Mess and Canteen Management System is a database-driven web application for a college hostel mess and canteen. It gives the **caterer** one system to run daily operations (menu, headcount, inventory, suppliers, staff), gives **students** a portal for leave, feedback, complaints and canteen ordering, and gives the **college** an independent record of meals served so the caterer's per-plate invoice can be verified before payment.

One-line summary: One database that links students, attendance, food, stock and money, so that headcount, invoices and inventory update automatically.

Problems addressed

- Meal counts are recorded manually, so caterer invoices are hard to verify.
- Caterers cook without a reliable headcount, causing wastage or shortages.
- Stock, purchases and supplier dues are tracked on paper.
- Student feedback and complaints have no tracking or resolution record.

Objectives

- Record every mess meal served and generate the caterer's invoice from that data.
- Forecast daily plates from enrolled students, approved leave and past turnout.
- Track inventory, purchases and supplier dues with automatic stock updates.
- Support canteen ordering, feedback and complaint resolution with audit trails.
- Demonstrate core DBMS concepts: normalization, triggers, procedures, views, transactions, indexing and access control.

2. Operating Model and Assumptions

Area	How it works	Status
Mess meals (students)	Free at point of use. Students are not billed monthly by the caterer. Only attendance is tracked.	Confirmed
Caterer payment	College pays the caterer **per plate served**. Invoice = sum of (plates x rate per meal type) for a period, generated from attendance.	Assumed, confirm with warden
Guest meals	Not billable. Not counted in invoices.	Confirmed
Hostel attendance	Existing manual ID-card scan at the warden's office, taken at night. Used for presence checks only. It is not meal attendance.	Confirmed
Mess attendance	Separate record captured at the mess entrance or serving counter (ID/QR scan or manual tick list) by a college-appointed person. This is the only basis for the invoice.	Proposed
Canteen	Students pay per order (cash, UPI or wallet).	Assumed, confirm

3. Users and Roles

Role	Responsibilities in the system
Student	View menus, request leave, place canteen orders, give feedback, raise complaints, view own attendance.

Role	Responsibilities in the system
Caterer / Manager	Plan menus, view headcount forecast, manage inventory, suppliers and staff, view invoices and cost reports, respond to feedback.
Mess / Canteen Staff	Mark mess attendance, bill canteen orders, enter stock usage and purchases.
Warden / College Admin	Approve leave, verify attendance and invoices, approve payment to the caterer, audit complaints, view reports.

4. Scope

In scope	Out of scope
Mess attendance, leave, headcount forecast Per-plate caterer invoice and approval workflow Canteen menu, ordering and order history Inventory, purchases, suppliers, wastage Feedback and complaint tracking Role-based dashboards and reports	Student monthly mess billing Online payment gateway integration Biometric or RFID hardware integration Staff payroll and salary processing Native mobile app

5. System Modules

#	Module	Purpose	Key tables
M1	Students, Enrollment and Leave	Student records, hostel mapping, mess enrollment, leave workflow	Hostel, Student, MessEnrollment, LeaveRequest
M2	Attendance	Meal slots, hostel presence, mess attendance, anomaly checks	MealSlot, HostelAttendance, MealAttendance
M3	Caterer Billing and Payments	Rates, invoice generation, verification, approval, payment	MealRate, CatererInvoice, InvoiceLine, CatererPayment
M4	Menu and Canteen Orders	Mess weekly menu, canteen items, orders and payment status	Category, MenuItem, MessMenu, Orders, OrderItem
M5	Inventory and Suppliers	Stock, purchases, consumption, wastage, supplier dues	InventoryItem, Supplier, Purchase, PurchaseItem, StockUsage
M6	Users, Feedback and Complaints	Login and roles, staff, ratings, complaint lifecycle	Staff, AppUser, Feedback, Complaint, ComplaintUpdate

Core data flow

Students enrolled (M1) -> approved leave reduces forecast (M1) -> meals marked at the mess (M2) -> plates counted per meal type -> invoice generated using the rate valid on each date (M3) -> college verifies against anomaly report -> approval and payment (M3). Menu (M4) drives consumption; stock usage and purchases (M5) give cost per plate; feedback (M6) rates each meal.

6. DBMS Concepts Demonstrated

Concept	Where it is applied	Owner
ER modelling (weak entities, 1:N, M:N resolution)	InvoiceLine depends on CatererInvoice; OrderItem and PurchaseItem resolve M:N relations	All, merged by M6
Normalization (1NF to 3NF)	Rates, menus and prices kept in separate tables; before/after decomposition documented	M1
Constraints (PK, FK, UNIQUE, CHECK, NOT NULL, ENUM)	UNIQUE(student, date, meal) blocks double attendance; CHECK on quantities and ratings	All
Joins, subqueries, aggregation, GROUP BY / HAVING	Plates per meal, cost per plate, top items, supplier spend	All

Concept	Where it is applied	Owner
Views	Daily plates, anomalies, low stock, daily canteen sales, invoice summary, open complaints	M2, M3, M4, M5, M6
Stored procedures	generate_invoice, place_order, record_purchase, approve_leave, resolve_complaint	M1, M3, M4, M5, M6
Triggers	Attendance validation and lock, stock update and low-stock flag, order total, invoice status order, complaint log	M1 to M6
Transactions (ACID, commit, rollback)	Invoice generation, order placement with payment, purchase entry	M3, M4, M5
Concurrency and locking	Two staff marking the same student; simultaneous stock deduction using row locks	M2, M5
Indexing and query optimization	Index on (meal_date, meal_type), (student_id), order date; EXPLAIN before and after	M3
Temporal data	MealRate with effective_from and effective_to so rate changes never alter past invoices	M3
Security and access control	Database users per role with GRANT and REVOKE; application role checks	M5
Auditing	Complaint and invoice status history tables	M3, M6

7. Team Roles and Responsibilities

Six members, six modules. Workload is balanced by complexity rather than table count. Each member is the single owner of their module from design to demo.

Equal workload standard (every member delivers all of the following)

Deliverable	Minimum per member
Database module	3 to 5 tables with keys, constraints and sample data
Programmed logic	At least 1 trigger and 1 stored procedure or transaction
Views and queries	At least 1 view and 4 non-trivial queries (joins, aggregates, subqueries)
Application	Backend routes and UI screens for own module
Cross-cutting duty	One shared task listed below
Documentation	One report chapter and two PPT slides

Member	Module	Cross-cutting duty
Member 1	M1 Students, Enrollment and Leave	Normalization documentation
Member 2	M2 Attendance	Sample data generation
Member 3	M3 Caterer Billing and Payments	Indexing and query performance
Member 4	M4 Menu and Canteen Orders	PPT and demo script
Member 5	M5 Inventory and Suppliers	Security and role-based access
Member 6 (Project Lead)	M6 Users, Feedback and Complaints	Master ER diagram, integration, final report

Member 1 - Students, Enrollment and Leave

Tables	Hostel, Student, MessEnrollment, LeaveRequest
Trigger	trg_leave_overlap: rejects overlapping leave requests for the same student
Procedure	sp_approve_leave(leave_id, approver): updates status and records the approver
View	v_students_on_leave_today
Sample queries	Students on leave by hostel; enrollment count per hostel; students with the most leave days; enrolled students with no meals marked
Application	Student registration and profile, leave request form, leave approval screen
Cross-cutting	Normalization: document 1NF, 2NF, 3NF decomposition with before and after tables
Report chapter	Requirements and Normalization

Member 2 - Attendance

Tables	MealSlot (meal type, timing, cutoff), HostelAttendance, MealAttendance
Trigger	trg_attendance_validate: blocks duplicates, entries outside the meal slot cutoff, and edits to already invoiced dates
Procedure	sp_mark_attendance(student_id, meal_type, staff_id)
Views	v_daily_plates, v_meal_anomalies (meal during approved leave, meal with no hostel record)
Sample queries	Plates per meal per day; turnout ratio per meal; flagged anomalies; hostel present but no meals
Application	Attendance marking screen, daily attendance report, hostel attendance import or entry
Cross-cutting	Sample data: about 200 students and 30 days of realistic attendance, leave and orders for the whole team
Report chapter	Attendance Design and Data Integrity

Member 3 - Caterer Billing and Payments

Tables	MealRate, CatererInvoice, InvoiceLine, CatererPayment
Trigger	trg_invoice_status: enforces the order Generated, Verified, Approved, Paid and stamps the user and time
Procedure	sp_generate_invoice(start, end): single transaction with overlap check, rate lookup by date, rollback on error
View	v_invoice_summary
Sample queries	Plates and amount per period; rate history; invoices by status; month-on-month plate trend
Application	Invoice generation screen, verification screen showing anomalies, approval and payment recording
Cross-cutting	Indexing: add indexes, run EXPLAIN before and after, report the improvement
Report chapter	Billing Logic and Transactions

Member 4 - Menu and Canteen Orders

Tables	Category, MenuItem, MessMenu, Orders (with payment mode and status), OrderItem
Triggers	trg_orderitem_price: copies the current price into the order line; trg_order_total: keeps the order total correct
Procedure	sp_place_order: one transaction for order, items and payment; rolls back if an item is unavailable
Views	v_daily_canteen_sales, v_top_items
Sample queries	Top 5 items per week; revenue per category; peak ordering hours; order history per student; items never ordered
Application	Mess weekly menu, canteen menu, cart and checkout, order history, menu management for the caterer
Cross-cutting	PPT and demo script: slide deck, demo flow and rehearsal schedule
Report chapter	Menu and Ordering

Member 5 - Inventory and Suppliers

Tables	InventoryItem, Supplier, Purchase, PurchaseItem, StockUsage
Triggers	trg_stock_on_purchase (adds stock); trg_stock_on_usage (deducts stock, rejects negative); low-stock flag at reorder level
Procedure	sp_record_purchase: transaction covering purchase, items and stock update
Views	v_low_stock, v_supplier_dues
Sample queries	Cost per plate (consumption cost divided by plates served); supplier-wise spend; wastage by item; items below reorder level
Application	Stock dashboard, purchase entry, daily usage entry, supplier dues screen
Cross-cutting	Security: database users per role with GRANT and REVOKE, plus a test showing denied access
Report chapter	Inventory and Access Control

Member 6 (Project Lead) - Users, Feedback and Complaints

Tables	Staff, AppUser (role), Feedback, Complaint, ComplaintUpdate
Trigger	trg_complaint_log: writes every status change into ComplaintUpdate
Procedure	sp_resolve_complaint(complaint_id, staff_id, note)
Views	v_meal_ratings, v_open_complaints_aging
Sample queries	Average rating per meal type per week; complaints unresolved for more than 3 days; lowest rated meals; staff count per role
Application	Login and role-based dashboards, feedback form, complaint tracker, backend project setup
Cross-cutting	Master ER diagram, merging module schemas, integration testing, final report compilation, schema freeze management
Report chapter	Introduction, ER Design, Integration and Conclusion

8. Timeline

Week	Milestone	Checkpoint
1	Confirm assumptions with the warden. Requirements per module. Module ER diagrams. Lead merges the master ER.	ER review
2	Relational schema, normalization, DDL scripts, sample data.	**Schema freeze** at week end

Week	Milestone	Checkpoint
3	Triggers, procedures, views, queries. Indexing and EXPLAIN analysis.	SQL scripts merged and run on one database
4	Backend and frontend per module. Role-based access. Integration.	End-to-end demo of the main flow
5	Testing, bug fixes, report, PPT, demo rehearsal.	Final submission

9. Deliverables

- Working web application (frontend, backend, database)
- ER diagram and relational schema
- SQL scripts: DDL, sample data, queries, views, triggers, procedures, access control
- Normalization document
- Project report and PPT
- Live demo covering: leave, attendance, invoice generation and approval, canteen order, stock update, complaint resolution

Suggested stack (confirm with faculty): MySQL or PostgreSQL, Python Flask or Node.js backend, HTML with Bootstrap or React frontend.

Team conventions

- One shared Git repository. Each member keeps SQL in their own file (for example sql/01_students.sql, sql/02_attendance.sql).
- Table names in PascalCase, columns in snake_case, primary keys named table_id.
- No schema change after the freeze without the Project Lead's approval.
- Weekly team meeting. Each member reports status against the checklist in section 7.

10. Open Questions to Confirm

#	Question	Who to ask	Impact
1	Is the caterer paid per plate, or a fixed monthly amount?	Warden or faculty	Invoice design
2	Is the rate the same for all meals, or per meal type? Is there a minimum daily plate guarantee?	Warden or caterer	MealRate and invoice logic
3	How will mess attendance be captured (scan at entrance or tick list) and by whom?	Warden	Attendance module
4	Is the warden's hostel attendance on paper, Excel or software?	Warden	Import or manual entry
5	Is there a canteen, and is it run by the same caterer? How do students pay?	Faculty or caterer	Scope of M4
6	What is the cutoff time for marking each meal?	Warden	MealSlot and trigger